

DAMA: Gap Analysis

Dual Agent Memory Architecture | 30 September 2026

Purpose: show what each related paper leaves open, then what DAMA proposes to address. Gaps are judged against the DAMA goal (managing the full memory lifecycle for another model, cheaply). They are not criticisms of the papers on their own terms. Based on each paper's published abstract and design.

Part 1. Gaps in each paper

Paper	What it does	Gap (for DAMA)
MemGPT (2023)	Pages memory in and out of context, like an operating system	The main (expensive) LLM manages its own memory. No separate, cheap controller.
Generative Agents (2023)	Stores experiences, retrieves them, reflects on them	Fixed retrieval scoring. Reflection needs many LLM calls. No real forgetting or updating of old facts.
LongLLMLingua (2023)	Compresses a long prompt before it is sent	Only shrinks what is already in the prompt. No long-term memory: nothing is stored, updated or forgotten.
SwiftSage (2023)	Small fast model plus large slow planner	Splits fast and slow thinking for actions, not for memory. Switching is rule-based.
LongMemEval (2024)	Benchmark for long-term chat memory	Measures answer accuracy only. Does not score token cost, latency, write quality or forgetting quality.
Zep (2025)	Time-aware knowledge graph as memory storage	Mainly a storage layer. Relies on LLM calls to build and update the graph. Does not decide what the main model should see or how many tokens it gets.
A-MEM (2025)	Linked notes that evolve when new memories arrive	Every new memory triggers LLM calls, which costs money and time. Forgetting is not a core operation.
Mem0 (2025)	Extracts, consolidates and retrieves key facts	A general LLM makes the memory decisions each turn. No small learned controller. No context budgeting.
Memory survey (2025)	Maps the whole field of agent memory	No system. Lists automated, learned, multi-agent and trustworthy memory as open problems.
MemRouter (2026)	Small learned router decides what to store	Covers only WRITE. Retrieval, update, merge, forget and budgeting are not handled. Still relies on a frozen 7B model underneath.
SAGE (2026)	Novelty gate: add, skip, or ask an LLM	Covers only WRITE. Novel does not always mean important. No retrieval, forgetting or budgeting.
Gated-Memory Routing (2026)	Learns which reasoning steps to keep for multi-agent tasks	Memory lasts one task, not across sessions. No handling of facts that change over time.
TypeSafe Jev (2026, industry)	Fast model that returns typed decisions, not text	Not a memory system. Speed and accuracy claims are the company's own, not independently checked.

Part 2. Common gaps across all papers

Gap	Closest existing work	What DAMA proposes
1. No single owner of memory. Storing, retrieving, updating and forgetting are handled by separate tools, prompts and scripts.	Mem0, A-MEM (partly)	One dedicated Memory Agent that owns the whole lifecycle.
2. Memory decisions are still made by large, general LLMs, which is slow and costly.	MemRouter, SAGE (write side only)	A small, fast, specialized model for all memory decisions, not just writing.
3. No control over how much context the main model receives.	LongLLMLingua (compression only)	A token budget per memory, set by the Memory Agent for the Primary Agent.
4. Forgetting is barely studied.	Mem0 (delete), SAGE (skip)	FORGET and EVICT as first-class decisions, with checks on quality.
5. Facts that change over time are handled only partly.	Zep (temporal graph)	UPDATE with history kept: old fact marked historical, new fact marked current.
6. No safety net when memory filtering is wrong.	SAGE (asks an LLM when unsure)	Confidence scores plus fallbacks: retrieve more, then a larger model, then raw history.
7. Evaluation looks at accuracy but not cost and memory quality together.	LongMemEval (accuracy)	Test with the same Primary Agent and swap only the memory. Measure success, tokens, latency, write precision and forgetting quality.

Part 3. What DAMA is solving

The problem: big models waste their context and budget on remembering. Existing research solves pieces of memory, not the whole job.

The idea: split the work. The Primary Agent thinks. A small Memory Agent remembers. The Memory Store keeps the data.

The new part: one learned controller for the whole memory lifecycle (write, retrieve, rank, update, merge, compress, forget, budget) working for another, larger model.

The expected benefit: fewer tokens sent to the big model, lower cost and delay, and more accurate, up-to-date memory.

The main risk: if the Memory Agent drops something important, the big model never sees it. Confidence scores and fallbacks are required.

Status: a proposal. Not built or tested yet. No DAMA results exist.

Central question: can a small, specialized model learn what a much larger model should remember, retrieve, update and forget, while using less context and losing no quality?

Sources: MemGPT 2310.08560, Generative Agents 2304.03442, LongLLMLingua 2310.06839, SwiftSage 2305.17390, LongMemEval 2410.10813, Zep 2501.13956, A-MEM 2502.12110, Mem0 2504.19413, Survey 2512.13564, MemRouter 2605.00356, SAGE 2605.30711, Gated-Memory Routing 2609.00237 (all arXiv); TypeSafe AI blog, 15 Sept 2026.